EXP 25

Q1.Create a program where multiple threads print numbers simultaneously

Java Code:

```
class NumberPrinter extends Thread {
    private int start;

    public NumberPrinter(int start, String name) {
        super(name);   // set thread name
        this.start = start;
    }

    public void run() {
        for (int i = start; i < start + 5; i++) {
            System.out.println(getName() + " prints: " + i);
        }
    }
}

public class MultiThreadNumbers {
    public static void main(String[] args) {
        NumberPrinter t1 = new NumberPrinter(1, "Thread-1");
        NumberPrinter t2 = new NumberPrinter(10, "Thread-2");
        NumberPrinter t3 = new NumberPrinter(20, "Thread-3");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Q2. Create a Java program using multiple threads to simulate bank transactions. One thread performs deposit operations, and another performs withdrawal operations, both printing transaction details simultaneously

Java Code:

```
class BankAccount {
   private int balance = 1000;

   public synchronized void deposit(int amount) {
      balance += amount;
      System.out.println(Thread.currentThread().getName() +
            " deposited: " + amount + " | Balance: " + balance);
   }

   public synchronized void withdraw(int amount) {
      if (balance >= amount) {
         balance -= amount;
         System.out.println(Thread.currentThread().getName() +
               " withdrew: " + amount + " | Balance: " + balance);
      } else {
         System.out.println(Thread.currentThread().getName() +
               " tried to withdraw " + amount + " | Insufficient balance!");
      }
   }
}

class DepositThread extends Thread {
   private BankAccount account;

   public DepositThread(BankAccount account, String name) {
      super(name);
      this.account = account;
   }
```

```
    public void run() {
        for (int i = 1; i <= 3; i++) {
            account.deposit(200);
        }
    }
}

class WithdrawThread extends Thread {
    private BankAccount account;

    public WithdrawThread(BankAccount account, String name) {
        super(name);
        this.account = account;
    }

    public void run() {
        for (int i = 1; i <= 3; i++) {
            account.withdraw(150);
        }
    }
}

public class BankTransactionDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        DepositThread depositThread = new DepositThread(account,
"DepositThread");
        WithdrawThread withdrawThread = new
WithdrawThread(account, "WithdrawThread");

        depositThread.start();
        withdrawThread.start();
    }
}
```

Q1 Output

```
Thread-1 prints: 1
Thread-2 prints: 10
Thread-3 prints: 20
Thread-1 prints: 2
Thread-2 prints: 11
Thread-3 prints: 21
Thread-1 prints: 3
Thread-2 prints: 12
Thread-3 prints: 22
Thread-1 prints: 4
Thread-2 prints: 13
Thread-3 prints: 23
Thread-1 prints: 5
Thread-2 prints: 14
Thread-3 prints: 24
```

Q2 Output

```
DepositThread deposited: 200 | Balance: 1200
WithdrawThread withdrew: 150 | Balance: 1050
DepositThread deposited: 200 | Balance: 1250
WithdrawThread withdrew: 150 | Balance: 1100
DepositThread deposited: 200 | Balance: 1300
WithdrawThread withdrew: 150 | Balance: 1150
```